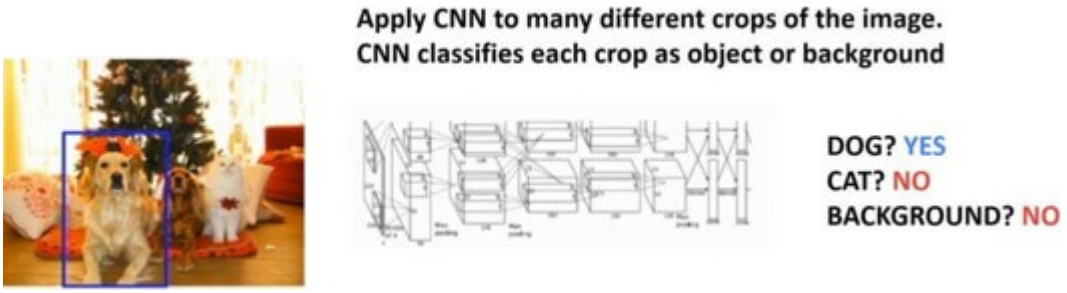


Highlights: Object detection is one of the most important tasks in Computer Vision. In this post, we will give an overview of one of the most influential families of object detection algorithms: R-CNN, Fast R-CNN, and Faster R-CNN. We will highlight the main novelties and improvements for each of them.

Finally, we will focus on the Faster R-CNN and explore the code and how it can be used in PyTorch.

Tutorial Overview:

1. Introduction to object detection
2. R-CNN
3. Fast RCNN
4. Faster RCNN
5. PyTorch implementation

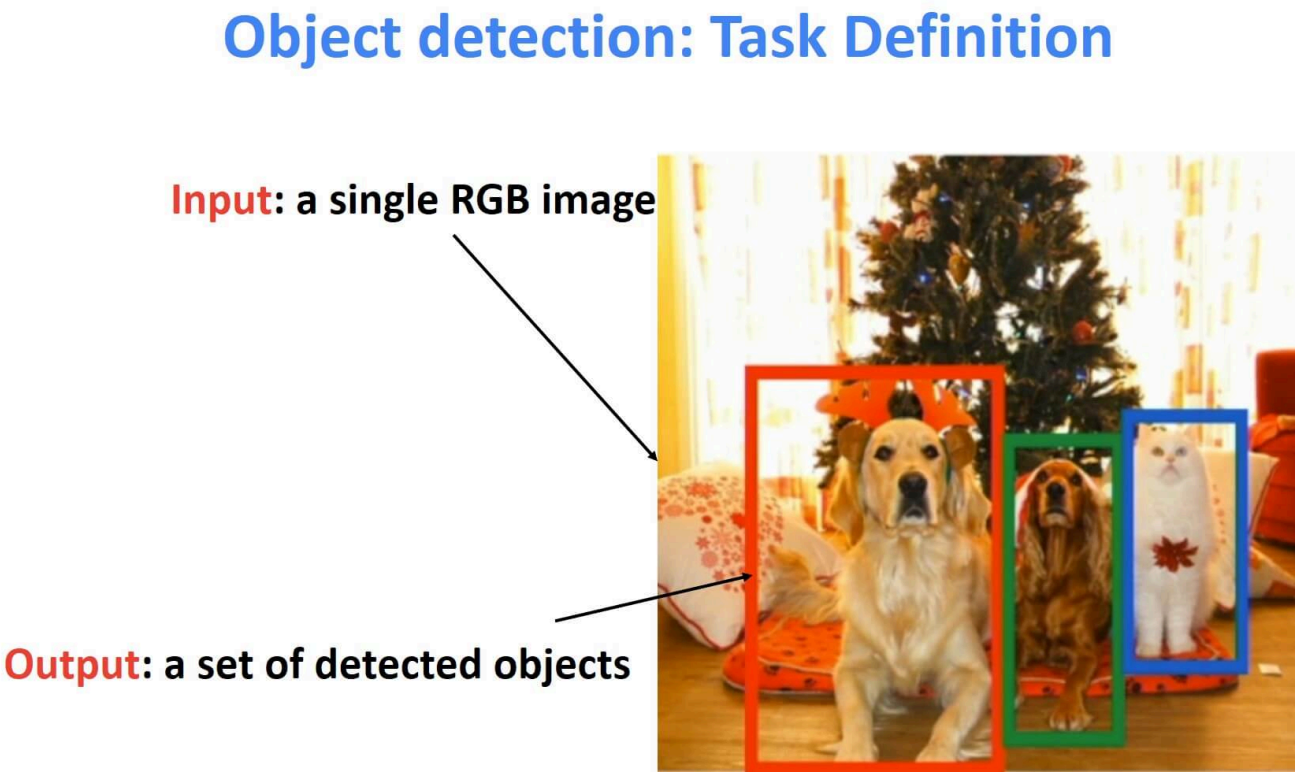
1. Introduction to object detection

The goal of object detection can be seen as an extension of the classification problem. In classification, we have one dominant instance or object in the image that occupies the central image area. Then, our goal is to detect what it is in the image.

Object detection is similar but more complex. First of all, we not only need to guess what it is in the image but also to detect its location. This is commonly done by positioning a rectangle box around the object-bounding box. This positioning is done with the respect to ground truth labeling. That is, the algorithm should learn the labeling process of the annotators and how exactly to place this rectangle box. In essence, this can be cast as a regression problem. On the other hand, the answer to what exactly in the image represents a classification problem.

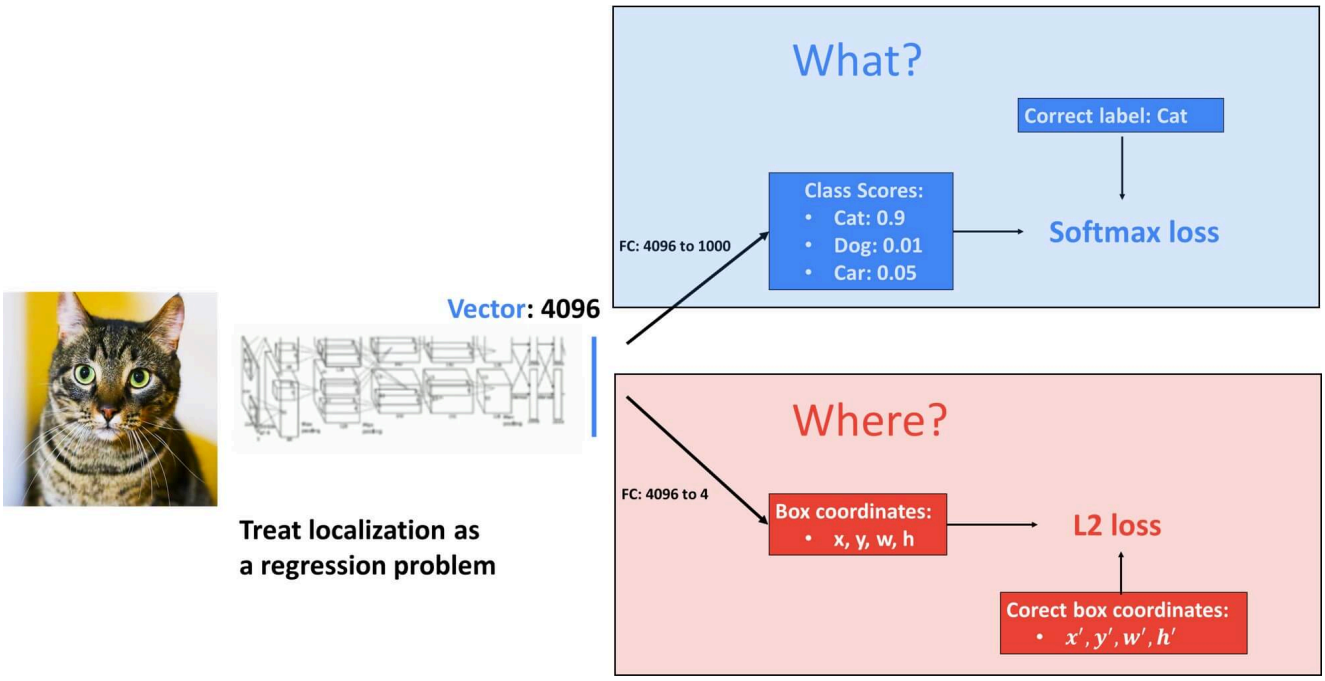
For each object we are predicting:

1. What is in the image? This is the classification task where we predict the category label
2. Where is the object in the image? This is the Localization task where we predict the bounding box around the object (4 numbers: x, y, width, height)



To better illustrate this, we will review the classification using neural networks. For instance, we can have an AlexNet.

Detecting single objects



This network will tell us "**What**" is in the image. For that, we will get an output score probability vector. For instance, there is a 0.9 chance that there is a cat in the image.

This prediction is obtained from the "last" fully connected layer of size 4096 and fed into a softmax activation function. Subsequently, this vector can also be used for the generation of the answer

"**Where**" a cat is in the image and this will be cast as the regression problem. Here, the regression output will be four values: **x, y, width, and height**.

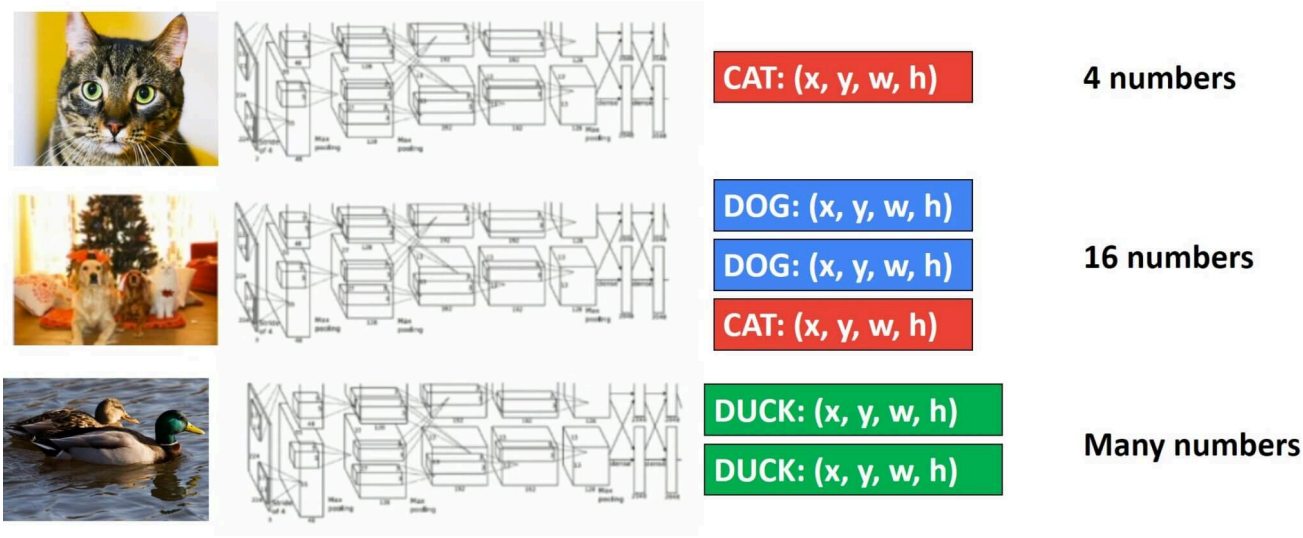
As for the loss function for classification, it is well-known that we will use a cross-entropy loss. On the other hand, for the regression problem, we will use an L2 loss function.

Now, the problem is that we have two different losses. However, we need a single loss function to apply gradient descent and optimize parameters. Well, the solution proves to be fairly simple. We will sum the two losses. That is, to be more precise, we will use a weighted sum and adjust the weight parameters. The weight of the parameters will imply what out of the two terms is more valuable to us. On the other hand, this can simply be used to adjust the scale for both losses.

Hence, we have one network and we want to output multiple results. This is standard architecture in Computer Vision and it is called a **Multitask Loss**.

However, there is one huge problem. We can have more than one object in the image, and now things are getting more complex 😳 So, we cannot exploit this idea of multitask loss.

Detecting multiple objects



In the Figure above, we can see an example of detecting a large number of "ducks" (or some other bird that is in the image).

2. R-CNN

A solution is to use sliding windows of different sizes and aspect ratios.

Have a look at the image below:

Sliding window